

• Contents: • Purpose:

- Measure PIC16F883 ADC behavior, then use the ADC result to control servo pulse width. The lab progresses from ADC bring-up to a 21-position servo, then to CCP-based timing with 6-bit and full 10 bit command resolution.

• Standards and references:

- RCET 3375 Lab Standard
- RCET PIC-AS Style Guide
- Starting a PIC-AS Project
- RCET 3373 PIC16F883 ADC and Sensor Conditioning
- RCET 3373 Lookup table and Dynamic Timing
- PIC16F882/883/884/886/887 Data Sheet
- PIC16F88X Family Silicon Errata and Data Sheet Clarifications

- Use the PIC16F883 data sheet as the device authority. Use the current ¹⁶ errata correction for the ADC acquisition-time calculation.

• Equipment and materials:

- PIC16F883 circuit with 4MHz crystal
- MPLAB X, pic-as, and PICkit 3
- potentiometer
- LEDs and current-limiting resistors for PORTB and PORTC
- digital multimeter and oscilloscope
- one servo for Part 2-4
- three servos and three potentiometers for optional Mastery
- suitable servo power supply, breadboard, and jumpers
- lab book

• Part 1 - ADC Proof of Life:

• Goal:

- Continuously convert a potentiometer voltage and make the complete 10-bit result observable. Use one analog input with VDD and VSS as the ADC references. Copy ADRESH to PORTC and ADRESL to PORTB after each conversion. Use one digital output to mark the conversion interval.

• Quick calculation references:

- Use the RCET 3373 ADC guide for the ADC model and the device data sheet for register details.
- For $F_{osc} = 4\text{MHz}$ and ADC clock $F_{osc}/8$:

$$T_{AD} = \frac{8}{4\text{MHz}} = 2\mu\text{s}$$

A 10-bit conversion uses 11 T_{AD} periods:

$$t_{conversion} = 11 T_{AD} \cdot 2\mu\text{s} = 22\mu\text{s}$$

potentiometer source resistance and acquisition time. Find the source resistance seen by the ADC, then use it in the acquisition timing. The ADC holding capacitor charges through the source and internal resistance.

$$\cdot 10\text{k}\Omega \text{ Pot } R_S = 5\text{k}\Omega \parallel 5\text{k}\Omega = 2.5\text{k}\Omega$$

- see next page for more.

- using the corrected errata example values at 25°C:

$$T_C = -(Hold(R_{IC} + R_{SS} + R_S) \ln\left(\frac{1}{2047}\right))$$

$$T_C = -(10\mu s)(1k\Omega + 7k\Omega + 2.5k\Omega) \ln\left(\frac{1}{2047}\right) \approx 0.80\mu s$$

- At 25°C, the temperature term is zero:

$$T_{ACQ} = T_{AMP} + T_C + T_{COFF}$$

$$T_{ACQ} = 2\mu s + 0.8\mu s + 0$$

$$T_{ACQ} = 2.8\mu s$$

- Use the actual potentiometer value, wiper position, and design assumptions for calculations.

• Volts per count -

- convert the measured reference range in ADC resolution. This gives the expected voltage represented by one ADC count

- For measured $V_{DD} = 9.96$ and $V_{SS} = 0V$:

$$\Delta V = \frac{9.96V}{1024} = 9.73mV/count \quad \Delta V = \frac{9.997V}{1024} = 9.76mV/count$$

- For ADC code 512:

$$V_{in} \approx 512 \cdot 9.73mV = 4.98V \quad V_{in} \approx 512 \cdot 9.76mV/count = 4.99V$$

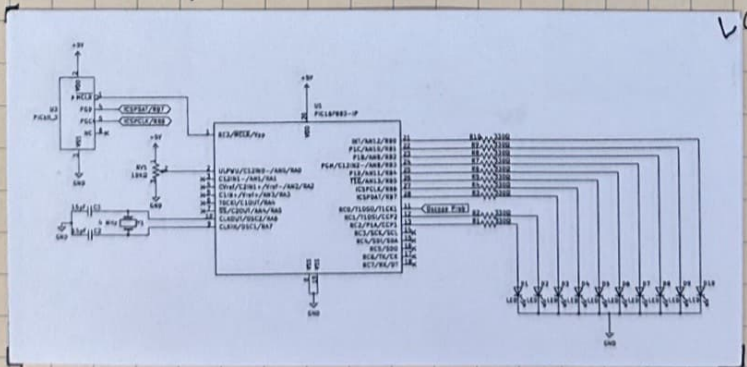
- use measure voltage from lab generator (4.99V)

• Before Lab:

- ADC/potentiometer schematic and electrical/loading analysis
- ADC SFR documentation for ANSEL, ANSELH, ADCON0, ADCON1, ADRESH, and ADRESL.
- selected channel, references, ADC clock, TAD, conversion time, source resistance, and acquisition time.
- main loop flowchart and source code.

• In the Lab:

- 1) Measure VDD and verify the potentiometer spans approximately 0v to vdd.
- 2) RVN continues conversions with calculated acquisition delay
- 3) Mark each conversion by driving the diagnostic output high immediately before setting GO/DONE and low after completion.
- 4) Measure the diagnostic pulse and compare it with the predicted conversion time. Account for instruction and polling overhead.
- 5) Demonstrate both right and left justified results on PORTB/PORTC.
- 6) Record DMM voltage and 10-bit ADC code near 0%, 25%, 50%, 75%, and 100% of the range.
- 7) Calculated voltage from each ADC code and compare it with the DMM measurement.



• Figure 1: Schematic ADC Proof of life.

• RAO has the pot to feed analog

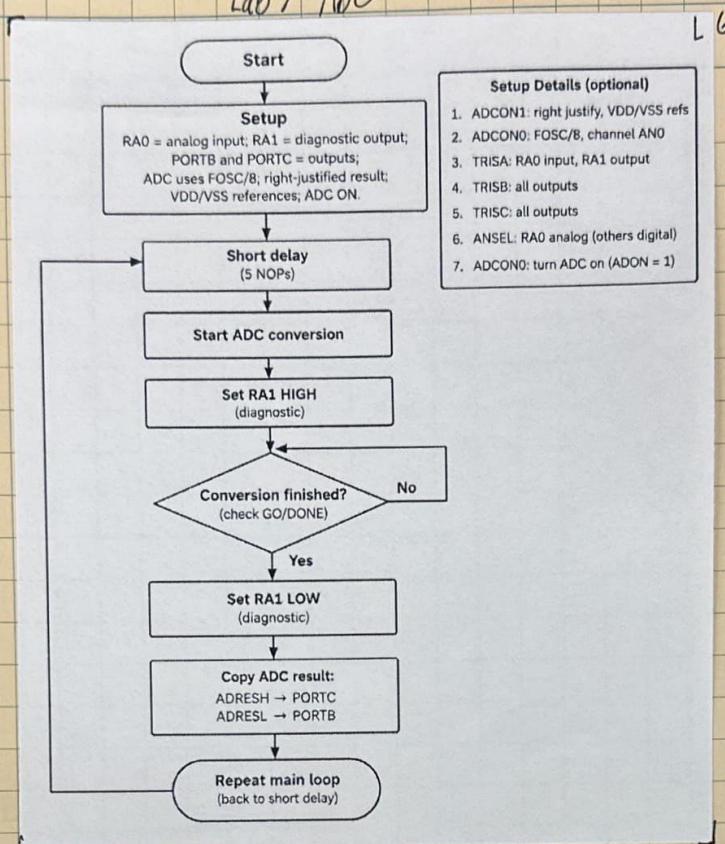


Figure 2: ADC Conversion and Result Display Flowchart

Setup:

L6

```

; Bank 1
BANKSEL TRISA
BSF    TRISA,0    ;Set RA0 as input for pot
BCF    TRISA,1    ;Set RA1 as output for diagnostic bit
CLRF   TRISB      ;Set PortB as outputs
CLRF   TRISC      ;Set PortC as outputs
BSF    ADCON1,7   ;Right justified
BCF    ADCON1,4   ;Voltage Reference bit for VDD
BCF    ADCON1,5   ;Voltage Reference bit for VSS
CLRF   ADRESL     ;Clear low register data

; Bank 3
BANKSEL ANSEL
BSF    ANSEL,0    ;Set RA0 as an Analog input
CLRF   ANSELH

; Bank 0
BANKSEL ADCON0
BCF    ADCON0,7   ;Clock Conversion selected bits Fosc/8
BSF    ADCON0,6   ;TAD = 2us
CLRF   ADRESH     ;clear high register data
CLRF   PORTB
CLRF   PORTC
BSF    ADCON0,0   ;Enable ADC bit
  
```

} ADC

} ADC needed

Figure 3: ADC Conversion Setup SFR's

Main:

```

NOP
NOP
NOP
NOP
NOP
NOP      ;delay
;      Start conversion
BANKSEL ADCON0
BSF      ADCON0,1 ;Set true A/D conversion status bit high which is automatically
               ;cleared by hardware when the conversion has completed

BANKSEL PORTA
BSF      PORTA,1 ;Set RAL high for diagnostic +1μs

```

Check:

```

BANKSEL ADCON0
BTFSC    ADCON0,1 ;check to see if its still converting
GOTO     Check
BANKSEL PORTA
BCF      PORTA,1 ;Set RAL low for diagnostic +1μs
;      Run the Diagnostic LOW if conversion is complete
BANKSEL ADRESH
MOVF     ADRESH,W
BANKSEL PORTC
MOVWF    PORTC ;upper 2 bits to PORTC

BANKSEL ADRESL
MOVF     ADRESL,W
BANKSEL PORTB
MOVWF    PORTB ;lower 8 bits to PORTB

GOTO     Main
End

```

Handwritten calculations:

Total time = $(2+1) \cdot 1\mu s = 3\mu s$
 $22\mu s + 3\mu s = 25\mu s$
 $22\mu s + 5\mu s = 27\mu s$

Figure 4: Main Code for ADC conversion

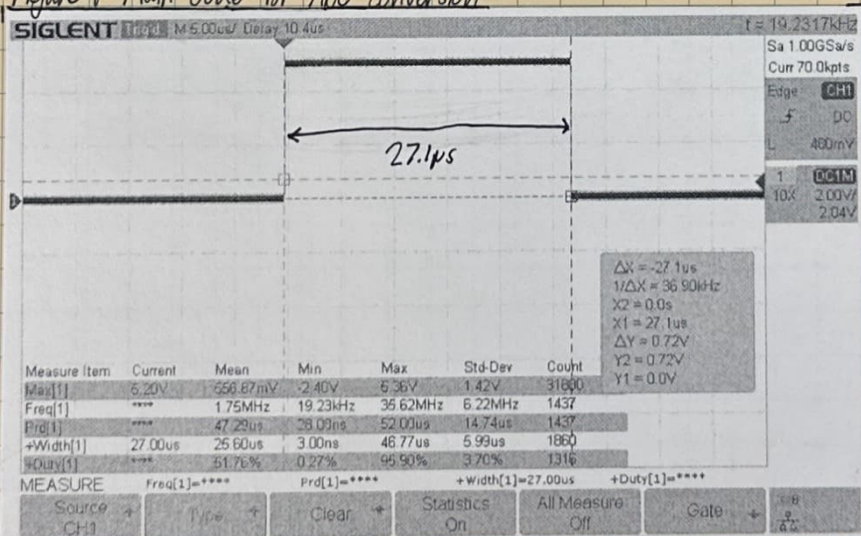


Figure 5: Waveform capture of Diagnostic bit

Measured Vin	ADC code	Voltage from ADC	Difference
0V	0	0V	0V
1.262V	256	1.249V	0.013V
2.515V	512	2.498V	0.017V
3.762V	768	3.747V	0.015V
4.991V	1023	4.991V	0V

$\Delta V = 4.879 \text{ mV/count}$
 $256 \cdot 4.879 \text{ mV} = 1.249 \text{ V}$
 $512 \cdot 4.879 \text{ mV} = 2.498 \text{ V}$
 $768 \cdot 4.879 \text{ mV} = 3.747 \text{ V}$
 $1023 \cdot 4.879 \text{ mV} = 4.991 \text{ V}$

Table 1: Measured - Voltage comparison table

Part 2 - ADC-to-Servo Mapping

Goal:

- Use the ADC result to command one servo through 21 discrete pulse widths from 500 μ s to 2.5ms in 100 μ s steps. Maintain a nominal 20ms frame. Verify the actual servo supply, control-input, current, and mechanical limits before connecting it.

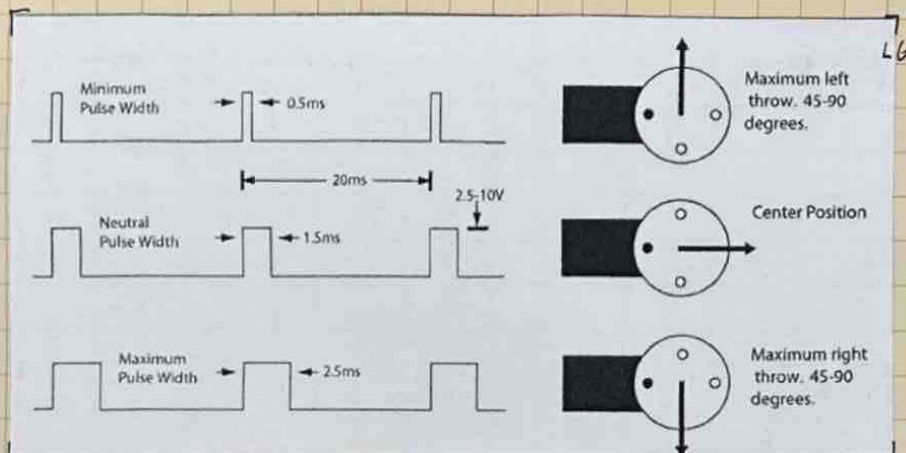


Figure 6: Servo Mapping for the AA⁶ ADC conversion

Quick calculation reference

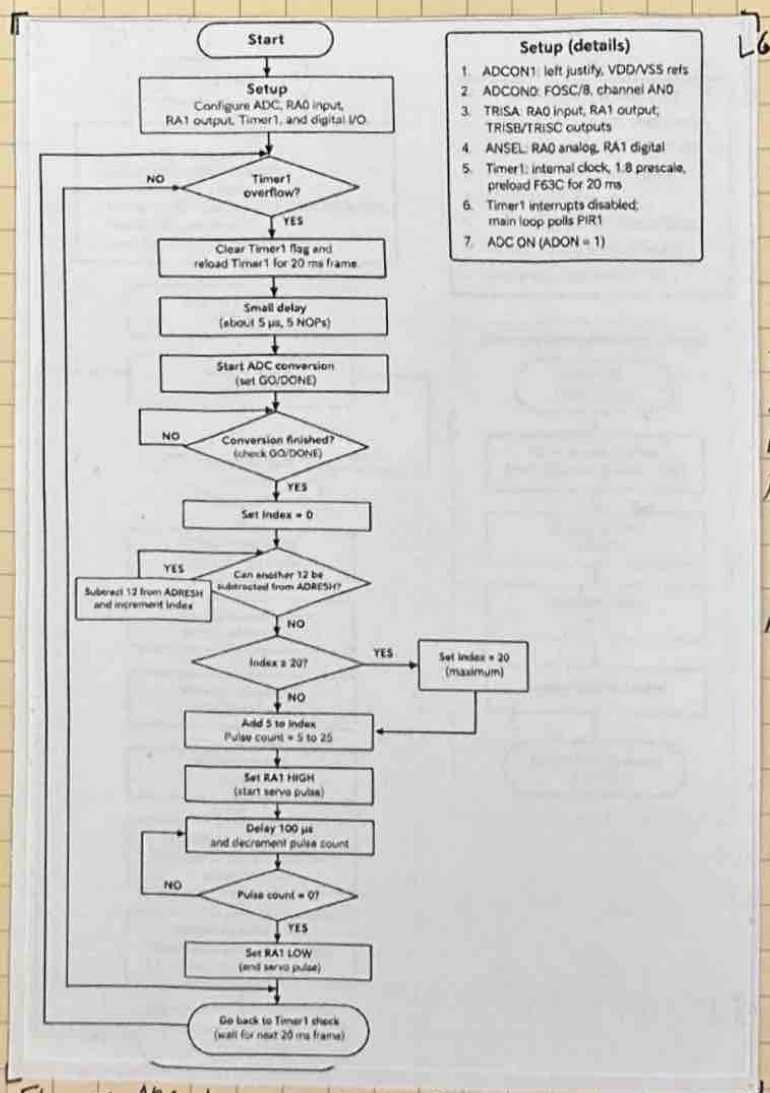
- determine the number of commanded positions. Both endpoints count as positions
 - span = $2.5ms - 0.5ms = 2.0ms$
 - intervals = $20ms / 0.1ms = 20$
 - positions = $20 + 1 = 21$
- for position index n , where $0 \leq n \leq 20$:
 - $t_{pulse} = 500\mu s + n(100\mu s)$
- example for the center command:
 - $t_{pulse} = 500\mu s + 10(100\mu s) = 1500\mu s$
- develop a monotonic mapping from the 10bit ADC range to indexes 0 through 20. All 21 indexes must be reachable

Before Lab:

- servo wiring and loading analysis
- ADC code ranges for all 21 positions⁶ index.
- expected pulse width for each index
- one hardware timer configured for a nominal 20ms frame.
- method for generating the variable high time
- calculations, flowcharts, and source code.

• In the lab: -keep the servo disconnected until instructor waveform checkoff.

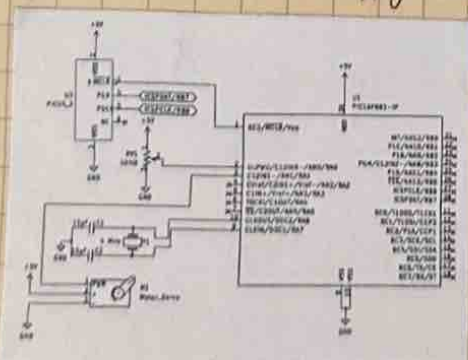
- 1) Verify the 20ms frame
- 2) Select each of the 21 indexes with the potentiometer and measure its High pulse width
- 3) Compare each measurement with the expected 500 μ s through 2.5ms sequence.
- 4) After checkoff, connect the servo with a suitable supply and common ground.
- 5) Sweep all 21 commands and record the observed mechanical position. Do not force the servo against a stop.



• The SFR's will be designed around keeping the code reusable

• The electrical analysis for the servo is pretty simple because the pic is only sending PWM and it is from + getting it main 5v from power supply. The supply voltage range for the HiTEC HS-322HD servo motor is 4.8v to 6v

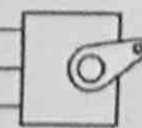
• Figure 7: ADC-to-servo Mapping flow chart



• Figure 8: ADC to servo Mapping schematic

• The servo won't be added until the waveform is approved with master Tim.

PWM = Orange (┌┐)
Vcc = Red (+)
Ground = Brown (-)



• Figure 9: Servo diagram

Index	ADC Range / Code	Expected Pulse	Measured Pulse	Observed Servo Position
0	0-47	500 μ s	544.998 μ s	0°
1	48-95	600 μ s	599.996 μ s	10°
2	96-143	700 μ s	699.992 μ s	20°
3	144-191	800 μ s	799.989 μ s	30°
4	192-239	900 μ s	899.986 μ s	40°
5	240-287	1.0 ms	999.983 μ s	50°
6	288-335	1.1 ms	1.099 ms	60°
7	336-383	1.2 ms	1.199 ms	70°
8	384-431	1.3 ms	1.299 ms	80°
9	432-479	1.4 ms	1.399 ms	90°
10	480-527	1.5 ms	1.499 ms	100°
11	528-575	1.6 ms	1.599 ms	110°
12	576-623	1.7 ms	1.699 ms	120°
13	624-671	1.8 ms	1.799 ms	130°
14	672-719	1.9 ms	1.899 ms	140°
15	720-767	2.0 ms	1.999 ms	150°
16	768-815	2.1 ms	2.099 ms	160°
17	816-863	2.2 ms	2.199 ms	170°
18	864-911	2.3 ms	2.299 ms	180°
19	912-959	2.4 ms	2.399 ms	190°
20	960-1023	2.5 ms	2.499 ms	200°

Table 2: 21 position verification table for verified servo control

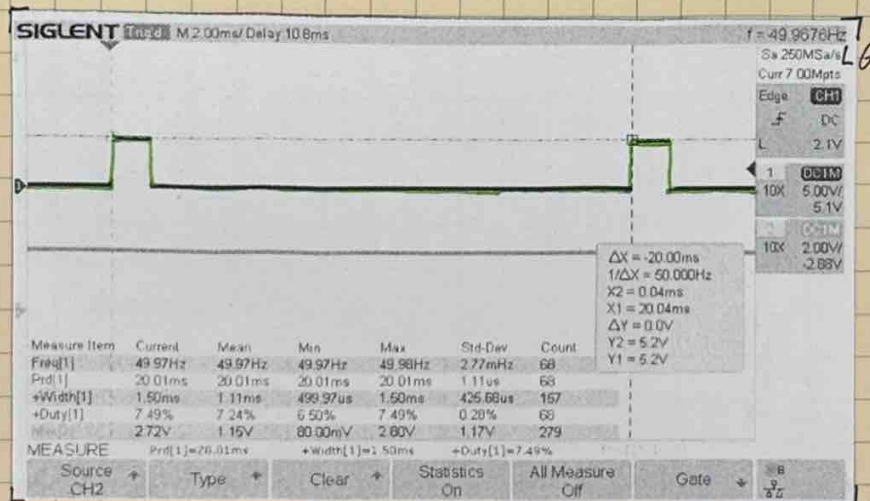


Figure 10: Waveform capture of the half way point 90° degrees

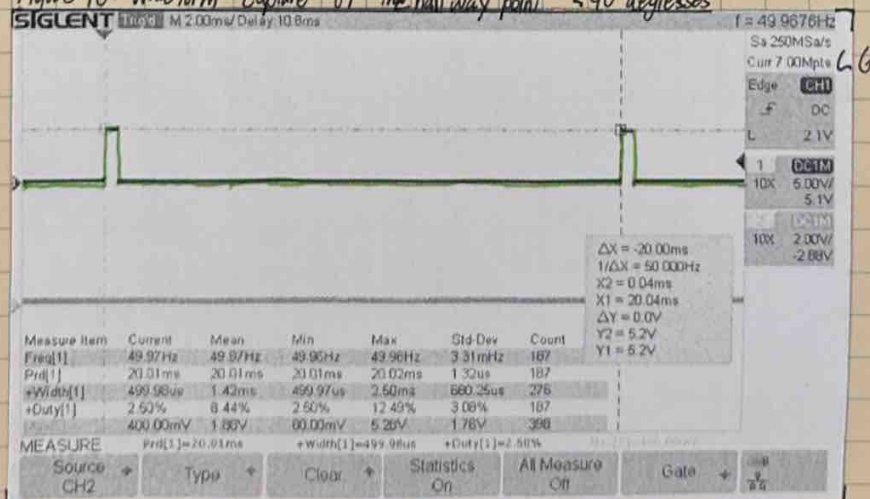


Figure 11: Waveform capture of the min point 20° degrees

```

Main:
    BANKSEL PIR1
    BTFSS PIR1,0      ;has Timer1 overflow happened
    GOTO Main          ;no keep checking, do nothing else
    BCF PIR1,0        ;yes handle it and run program
    BANKSEL TMR1H      ;reload Timer1 for the next 20ms delay set
    MOVLW 0xF6
    MOVWF TMR1H
    MOVLW 0x3C
    MOVWF TMR1L
    NOP                ;short pause so the ADC can settle on the
    NOP                ;pot's voltage before we lock it in
    NOP
    NOP
    NOP
    BSF ADCON0,1       ;turn on A/D conversion status bit

Conversion:
    BTFSC ADCON0,1     ;keep checking until it clears itself when done
    GOTO Conversion    ;
    CLRF Index         ;clear index to zero for the next section

IndexLoop:
    MOVLW 12           ;move 12 to be subtracted from 255
    SUBWF ADRESH,F     ;subtract 12 from ADRESH until 0 or negative value
    BTFSS STATUS,0     ;did ADRESH go negative? yes then GOTO IndexDone
    GOTO IndexDone     ;indexdone checks to see if the index is in a valid range
    INCF Index,F       ;if still a positive count then increment index until 0
    GOTO IndexLoop     ;stay in the loop until the count is zero or negative.

IndexDone:
    MOVLW 20           ;checks to make sure in desired range
    SUBWF Index,W      ;W = Index - 20
    BTFSS STATUS,0     ;Carry = 0 if Index is less than 20
    GOTO MakePulse     ;Index is 0-19, so keep it
    MOVLW 20           ;20 is the max limit of index count
    MOVWF Index        ;Index was 20 or 21, so limit it to 20

; turn the position number into an actual pulse

MakePulse:
    MOVLW 5            ;5 will be added to index
    ADDWF Index,F      ;Index is how many 100us for PWM
    BSF PORTA,1        ;start the pulse RAL goes HIGH here

PulseLoop:
    CALL Delay         ;call the delay section for a 100us delay
    DECFSZ Index,F     ;do the delay the Index amount of times
    GOTO PulseLoop     ;keeps going until index is zero
    BCF PORTA,1        ;once zero set RAL low
    GOTO Main          ;return

; Delay is producing a 100us

Delay:
    MOVLW 30           ;run the DelayCount 30 times
    MOVWF DelayCount   ;move 30 into DelayCount
    NOP                ;1 Tcy
    NOP                ;1 Tcy

DelayLoop:
    DECFSZ DelayCount,F ;Decrement from 30 until zero for the delay
    GOTO DelayLoop     ;keeps looping until DelayCount is zero
    RETURN             ;Return to PulseLoop
End

```

Figure 12: Main Code for Servo Mapping

10/6/2026

LR6

115

Setup:

```

; Bank 1
BANKSEL TRISA
BSF TRISA,0 ;RA0 is an input - this is where the pot wiper connects
BCF TRISA,1 ;RA1 is an output - this is the wire that goes to the servo
BCF ADCON1,7 ;left justified results this time
BCF ADCON1,4 ;top of the ADC's measuring range = VDD
BCF ADCON1,5 ;bottom of the ADC's measuring range = VSS (ground)
; Bank 3
BANKSEL ANSEL
BSF ANSEL,0 ;set RA0 is a analog input pin
BCF ANSEL,1 ;set RA1 as a digital I/O pin
CLRF ANSELH
; Bank 0
BANKSEL ADCON0
BCF ADCON0,7 ;pick the ADC's internal speed
BSF ADCON0,6 ;Fosc/8
CLRF ADRESH
CLRF ADRESL
CLRF PORTA ;make sure nothing is on port a
BSF ADCON0,0 ;turn the ADC module on
MOVLW 0xF6
MOVWF TMR1H ;load the timer's starting count, high byte
MOVLW 0x3C
MOVWF TMR1L ;low byte, so it takes exactly 20ms to roll over
BCF PIR1,0 ;make sure the timer's flag starts out clean
BANKSEL PIE1
BCF PIE1,0 ;let Timer1 actually trigger an interrupt when it rolls over
BANKSEL T1CON
MOVLW 0b00110001 ;prescale 1:8, use the chip's own clock, and start the timer
MOVWF T1CON
BCF PIR1,0 ;clear Timer1 overflow interrupt flag bit

```

L6

*Figure 13: SFR Servo Mapping

L6

10-7-26

- its a ruff resolution in the steps and its very jittery. It is around 10° per resolution.

L6

10/6/2026

LR6

Part 3-6 Bit Lookup table and CCP Compare

Goal: Increase command resolution to 64 values. Use the upper 6 ADC bits as an index, timer1 as a 1 μ s timebase and the 20ms frame, and CCP1 Compare, to schedule the servo falling edge. For this part, timer1 reloads to 0xB1E0 for the required 20ms timing at each frame state. The supplied lookup table module is built for that reload and 1 μ s timer tick.

CCP Compare quick reference: Timer1 provides the running 16 bit timebase. CCP1H:CCP1L stores a 16-bit compare value. When timer1 matches that value, CCP1 set CCP1IF. Use the compare interrupt to drive the servo output low so the CPU doesn't need to wait in a software delay for the falling edge and there will be only one interrupt at the end of the pulse.

- Document T1CON, TMR1H:TMR1L, CCP1CON, CCP1H:CCP1L, PIR1:CCP1IF, PIE1:CCP1IE, and any other required SFR's from the PIC16F883 datasheet.

6-bit ADC result to 16bit CCP1 compare value mapping: There are 64 commands and 63 intervals

$$\Delta t = \frac{2500\mu s - 500\mu s}{63} = 31.746\mu s$$

Only two right shifts are required because ADRESH already contains the upper eight ADC bits. The resulting six-bit index is ADC bits 9:4. For index 32:

$$t_{32} = 500\mu s + \text{round}\left(\frac{32(2000\mu s)}{63}\right)$$

$$t_{32} = 500\mu s + 1016\mu s = 1516\mu s$$

- Timer1 starts at 0xB1E0 = 45536 and ticks every 1 μ s

$$\text{CCP match} = 45536 + 1516\mu s = 47052$$

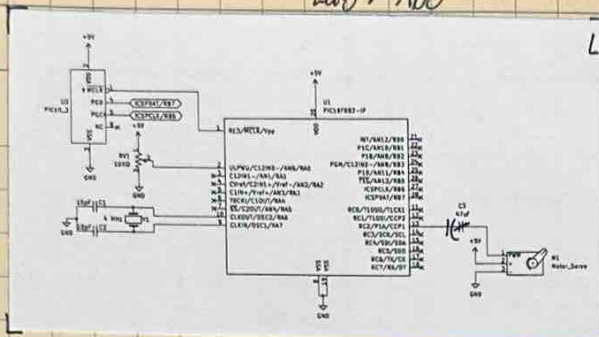
- See ross timo github for all RCET 3375/Lab Assignments/Lab7-ADC.md for the Required Ram, Required setup, and Program structure.

Before Lab:

- Timer1 1 μ s tick and 0xB1E0 reload calculation. Verify the 20ms frame time
- CCP1 SFR documentation
- left justified ADRESH to 6 bit index reduction and one worked lookup table⁶⁶ calculation.
- supplied lookup table module added to the project
- main/ISR flowcharts and source code

In the Lab:

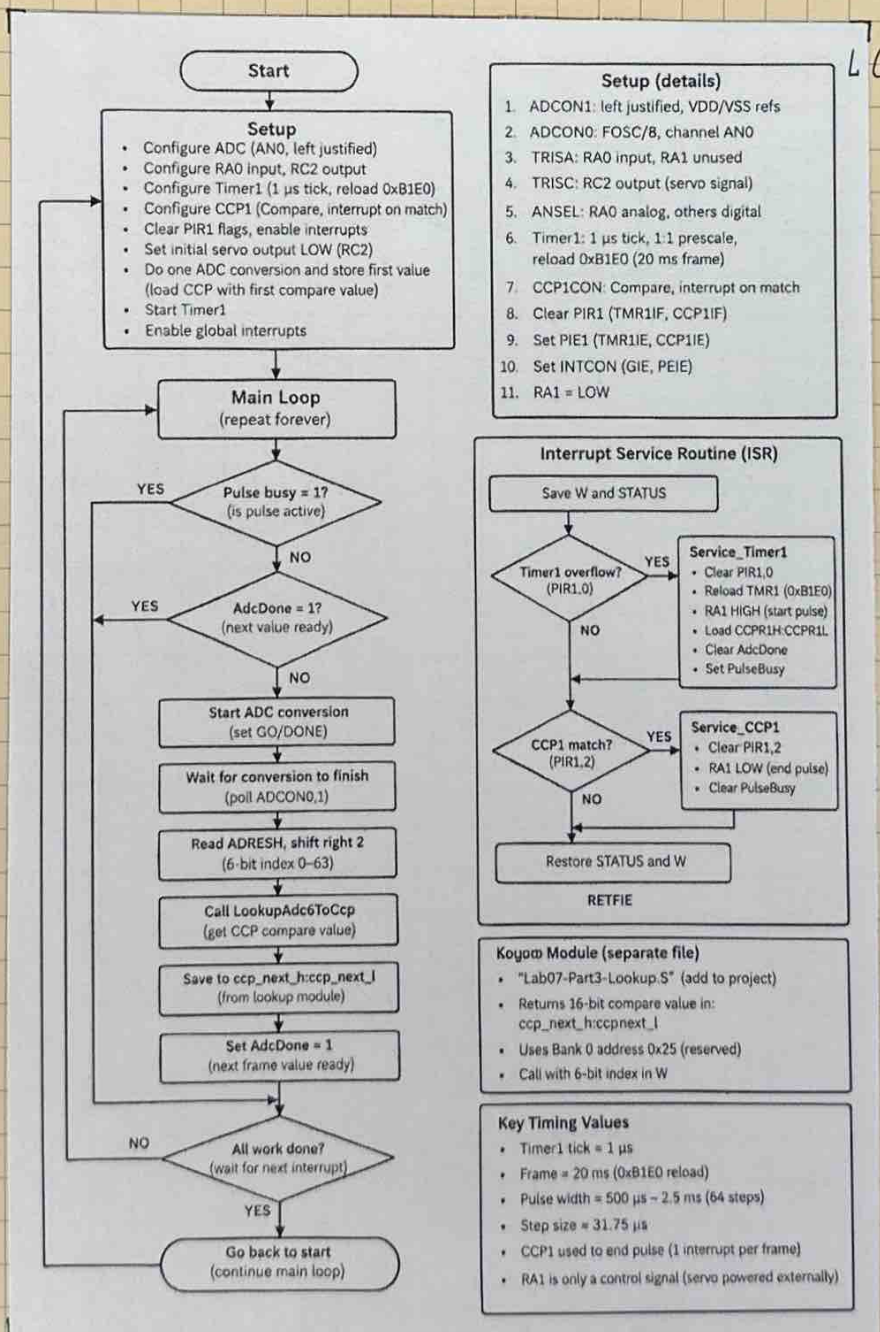
- 1) Verify Timer1 frame timing and CCP interrupt operation with the servo disconnected.
- 2) Verify first, center, and last table entries, then several intermediate values.
- 3) Confirm adjacent table entries differ by 31 or 32 μ s, averaging approximately 31.75 μ s across the full range.
- 4) Obtain instructor waveform checkoff
- 5) Connect the servo and sweep the 64 commands
- 6) Observe the mechanical response and compare it with Part2 resolution. Is the servo resolution noticeable? Is the servo movement smooth?



L6

The capacitor is to help smooth things out.

Figure 14: Full schematic for Part 3 6-bit Lookup table



L6

Figure 15: Flow chart for Part 3 with processes for 6-bit lookup table and CCP Compare

Setup:

```

: Bank 3
BANKSEL ANSEL
BSF ANSEL,0 ;set RA0 is a analog input pin
CLRFB ANSELH
: Bank 1
BANKSEL TRISA
BSF TRISA,0 ;RA0 is an input - this is where the pot wiper connects
BCF TRISC,2 ;RC2 is an output to the servo for control
BCF ADCON1,7 ;left justified
BCF ADCON1,4 ;top of the ADC's measuring range VDD
BCF ADCON1,5 ;bottom of the ADC's measuring range VSS (ground)
: Bank 0
BANKSEL ADCON0
BCF ADCON0,7 ;pick the ADC's internal speed
BSF ADCON0,6 ;Fosc/8
CLRFB ADRESH
CLRFB ADRESL
BSF ADCON0,0 ;turn the ADC module on
MOVLW 0b00001010 ;CCP1 in Compare mode: just flag an interrupt on
MOVWF CCP1CON ;match, don't touch the pin directly ourselves
MOVLW 0xB1
MOVWF TMR1H ;load the timer's starting count, high byte
MOVLW 0xE0
MOVWF TMR1L ;low byte, so it takes exactly 20ms to roll over
BCF PORTC,2 ;force the servo low so no weird glitch
BCF PIR1,0 ;clear the timer1 interrupt flag
BCF PIR1,2 ;clear the CCP1 interrupt flag
BANKSEL PIE1
BSF PIE1,0 ;Timer1 CAN now trigger an interrupt when it rolls over
BSF PIE1,2 ;CCP1 can now trigger an interrupt on a compare match
BANKSEL ADCON0
CLRFB Pulse ;nothing is happening yet
CLRFB ADCDone

```

LG

Figure 16: Setup code for 6-bit Lookup table and CCP Compare interrupts.

StartupWait:

```

BTFSC ADCON0,1
GOTO $-1
MOVFB ADRESH,W
MOVWF ADCIndex
BCF STATUS,0
RRFB ADCIndex,F ;shift right once RRF rotates f right through a carry flag
BCF STATUS,0
RRFB ADCIndex,F ;again so ADCIndex is now a clean 0-63 number
MOVFB ADCIndex,W
PAGESEL LookupAdc6ToCcp ;the lookup routine might live on a different
CALL LookupAdc6ToCcp ;page of program memory, so this makes sure
PAGESEL $ ;the CALL actually lands on it
MOVFB ccp_next_h,W
MOVWF CCP1RH ;manually load the very first pulse's end-time,
MOVFB ccp_next_l,W ;since Timer1's ISR (which normally does this)
MOVWF CCP1RL ;hasn't run yet
BSF ADCDone,0 ;this value is ready and waiting
MOVLW 0b00000001 ;prescale 1:1, internal clock, and start Timer1
MOVWF T1CON ;counting right now, from 0xB1E0
BSF INTCON,6 ;allow peripherals to interrupt at all
BSF INTCON,7 ;interrupts can now actually happen

```

LG

Main:

```

BTFSC Pulse,0 ;is a pulse still actively running
GOTO Main ;if high don't touch the ADC yet
BTFSC ADCDone,0 ;have we already prepared next frame's value
GOTO Main ;if high then nothing more to do until the next frame

```

Figure 17: Setup continued for the Lookup ADC6ToCcp give lookup table

```

BSP      ADCON0,1
ConvWait:
BTFSC   ADCON0,1
GOTO    ConvWait
MOVWF   ADRESH,W
MOVWF   ADCIndex
BCF      STATUS,0
RRF      ADCIndex,F
BCF      STATUS,0
RRF      ADCIndex,F      ;ADCIndex is now 0-63
MOVWF   ADCIndex,W
PAGESEL LookupAdc6ToCcp
CALL    LookupAdc6ToCcp  ;ccp_next_hiccp_next_1 now holds next
PAGESEL 0                ;frame's pulse end-time
BSP      ADCDone,0        ;done for this frame - go idle
GOTO    Main

; ISR - Timer1 starts each frame's pulse, CCP1 ends it. Nothing else
; in this program can interrupt, so either one could have fired.

IsrHandler:
MOVWF   W_TEMP
SWAPF   STATUS,W
MOVWF   STATUS_TEMP
BTFSC   PIR1,0            ;did Timer1 just roll over? (new frame)
CALL    Service_Timer1
BTFSC   PIR1,2            ;did CCP1 just match? (pulse should end)
CALL    Service_CCP1
SWAPF   STATUS_TEMP,W
MOVWF   STATUS
SWAPF   W_TEMP,F
SWAPF   W_TEMP,W
RETFIE

```

Figure 18: Conversion and rotating bits right to make the 6 bit register & ISR handler

```

; Service_Timer1: a new 20ms frame has begun. Start the pulse,
; tell CCP1 when to end it, and clear the way for main to start
; preparing the NEXT frame's value.

Service_Timer1:
BCF      PIR1,0            ;clear the flag
MOVLW   0xB1
MOVWF   TMR1H              ;reload the same starting count
MOVLW   0xE0
MOVWF   TMR1L              ;so every frame is exactly 20ms

BSP      PORC,2            ;pulse starts now - servo pin goes HIGH

MOVWF   ccp_next_h,W
MOVWF   CCPR1H              ;tell CCP1 the exact Timer1 count that
MOVWF   ccp_next_1,W        ;should end THIS pulse (calculated last frame)
MOVWF   CCPR1L

BCF      ADCDone,0          ;main is now free to prepare next frame's value
BSP      Pulse,0           ;and main must NOT touch the ADC until this clears
RETURN

; Service_CCP1: Timer1's count just reached the target we loaded -
; this pulse is over.

Service_CCP1:
BCF      PIR1,2            ;clear the flag
BCF      PORC,2            ;pulse ends now - servo pin goes LOW
BCF      Pulse,0           ;main is now allowed to work again
RETURN

End

```

Figure 19: ISR + Servicing routines for Timer 1 and CCP1

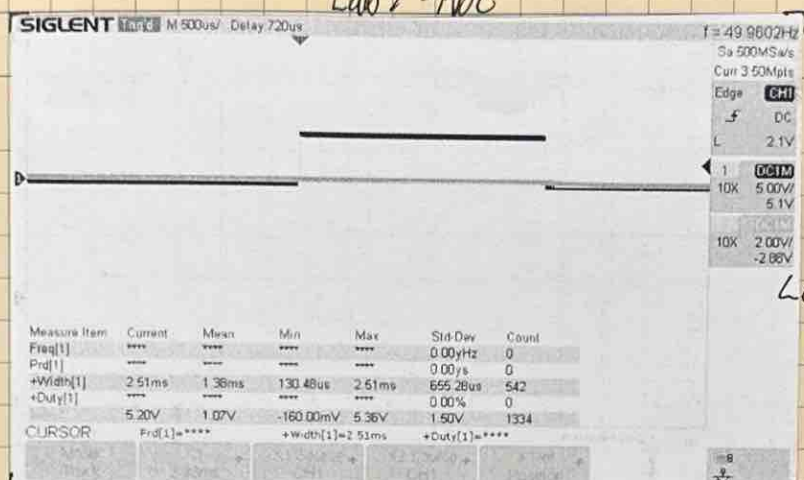


Figure 20: Oscilloscope capture of max PWM FCounter = 2514.921 μ s

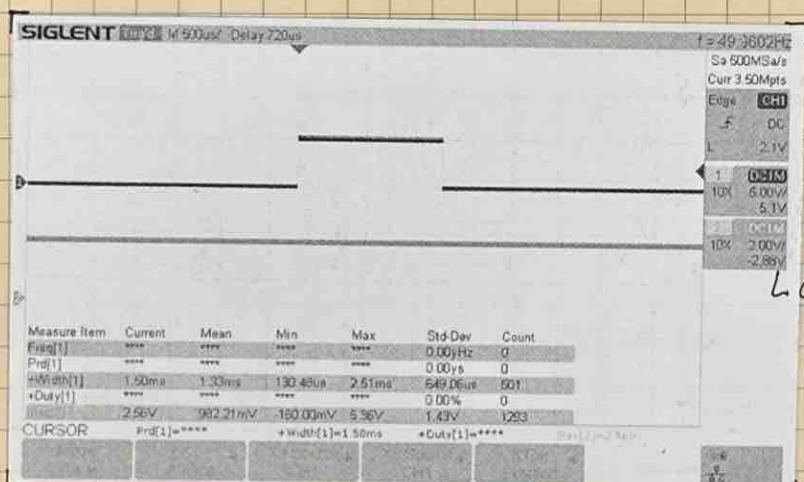


Figure 21: Oscilloscope Capture of middle range PWM FCounter = 1498.95 μ s

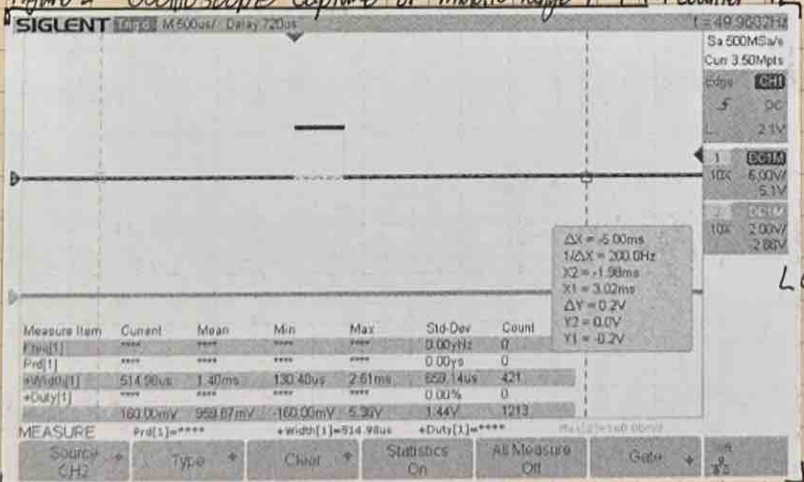


Figure 22: Oscilloscope Capture of minimum PWM - FCounter = 2514.921 μ s

Timer 1 counts every 1ps. Every 20ms it rolls over causing the new cycle to run. CCP1 waits and compares Timer 2 live count against the give number to get the count once matched it throws the interrupt. Main's job is reading the pot and turning that into the NEXT frame CCP target. Observed way less jitter on the servo and about 3.6 to 4 degrees per step. With 32.1 μ s resolution per step.

Lab 7 - ADC

Part 4 - Full 10 Bit Calculated Mapping:

Goal: Using lookup tables to increase servo ⁶ PWM resolution quickly becomes impractical. Use all 10 ADC bits without two 1024-entry lookup tables by replacing the 6-bit lookup table mapping with a full 10-bit mapping calculation. Keep the Timer1/CCP frame and pulse architecture from Part 3, use a right justified 10-bit ADC result.

- 10-bit ADC result to 16-bit CCP4 compare value mapping: Example

• The ideal linear mapping is:

$$t_{pulse} = 500\mu s + \frac{ADC(2000\mu s)}{1024}$$

• The supplied mapping routine uses this PIC-friendly integer approximation:

$$t_{pulse} = 500\mu s + 2(ADC) - \left[\frac{3(ADC)}{64} \right]$$

$$t_{pulse} = 500\mu s + 2(512) - \left[\frac{3(512)}{64} \right]$$

$$t_{pulse} = 500\mu s + 1024 - 24$$

$$t_{pulse} = 480 \text{ } ^{6} 1500\mu s$$

• With Timer1 starting at $0x81E0 = 45536$:

$$CCP \text{ match} = 45536 + 1500 = 47036$$

- The supplied mapping module uses the stored 10-bit ADC result in `adc_h:adc_l = 0x0200` and returns the 16-bit absolute compare value in `ccp_next_h:ccp_next_l = 0xB7BC` providing the falling edge timing for the next frame.

- Supplied mapping Mode: See link part 4 = <https://github.com/rossimo/RCJET3375/blob/main/Lab.../Lab7ADC.c>

• Required RAM: Same as part 3 but with a stored 2-byte ADC result instead of a 6-bit index.

• Required setup: Same as part 3 but with a right justified ADC result instead of left justified.

• Program Structure: Keep the same one update per frame sequence from part 3.

Before Lab:

• Part 3 Timer1/CCP design

• right justified 10-bit ADC result, integer mapping, endpoint check, and one worked example.

• supplied mapping module added to the project.

• main/ISR flowcharts and source code.

In the Lab:

1) Verify Timer1 frame timing and CCP interrupt operation with the servo disconnected.

2) Verify low, center, and high ADC commands, then several intermediate values.

3) Confirm adjacent ADC codes change the calculated pulse width by 1 or 2 μs , averaging approximately 1.96 μs across the full range.

4) Obtain instructor waveform checkoff.

5) Connect the servo and sweep the 1024 commands

6) Observe the mechanical response and compare it with Part 2 and Part 3 resolution. Is the servo resolution noticeable? Is the movement smooth?

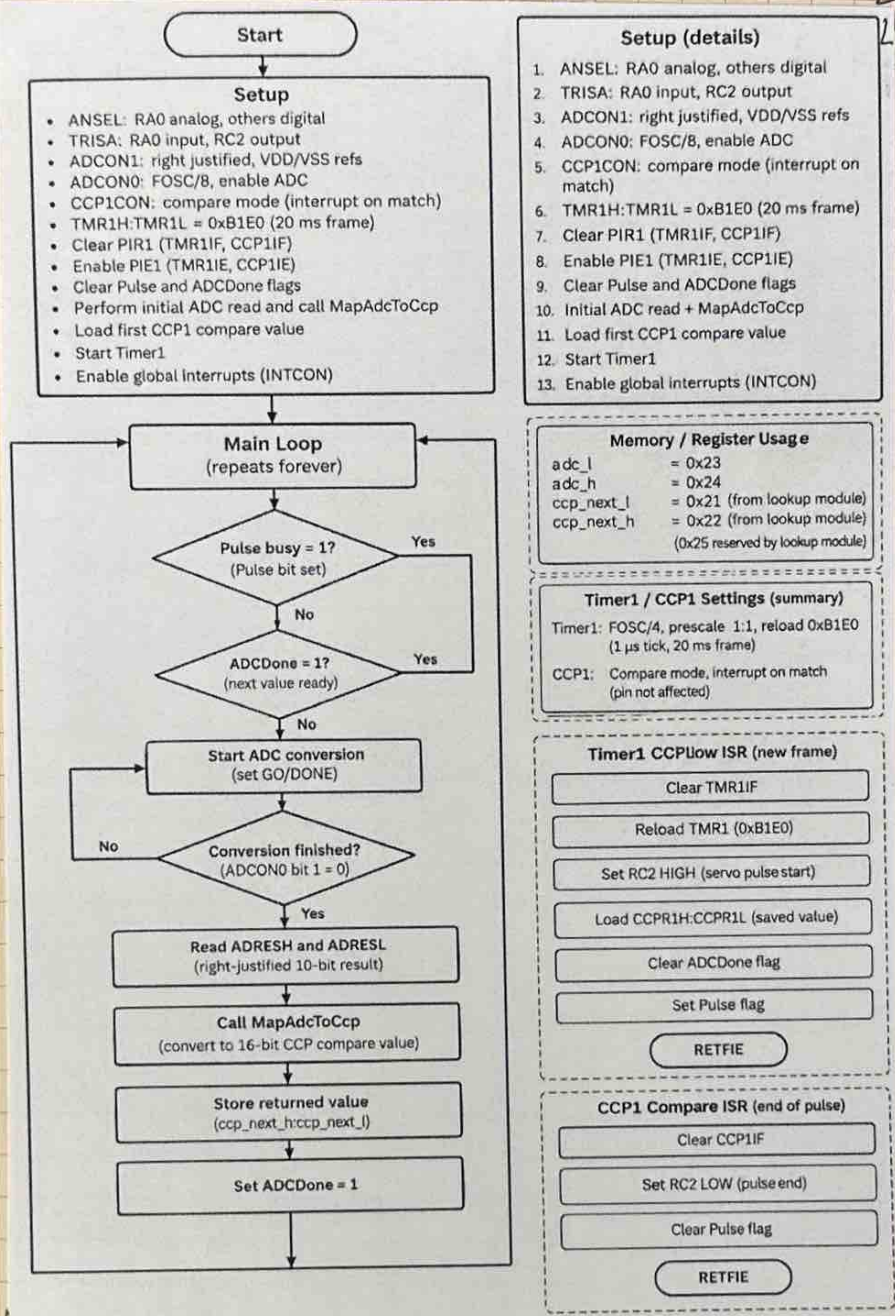


Figure 23: Full 10-Bit Mapping Flow chart for Part 4 with 1024 resolution

```

#include <xc.h>

ccp_next_l EQU 0x21 ;lookup module writes the answer here (low byte)
ccp_next_h EQU 0x22 ;...and here (high byte) - 0x25 is reserved, don't touch it
adc_l EQU 0x23 ;
adc_h EQU 0x24 ;

GLOBAL ccp_next_l, ccp_next_h, adc_l, adc_h
EXTERN MapAdcToCcp ;this is the called routine

W_TEMP EQU 0x70 ;save W during the ISR
STATUS_TEMP EQU 0x71 ;save STATUS during the ISR
Pulse EQU 0x72 ;bit0: 1 while the servo pulse is actively HIGH
ADCDone EQU 0x73 ;bit0: 1 once this frame's NEXT value is ready
ADCIndex EQU 0x74 ;the ADC reading, squeezed down to 0-63
  
```

Figure 24: The include EQU Registers for required mapping and code.

StartupWait:

```

BTFSC  ADCON0,1
GOTO   $-1
BANKSEL ADRESH
MOVWF  ADRESH,W
MOVWF  adc_h      ;full 10-bit result, right-justified, no shifting needed
BANKSEL ADRESL
MOVWF  ADRESL,W
BANKSEL adc_l
MOVWF  adc_l      add/changed from part 3

PAGESEL MapAdcToCcp ;the mapping routine might live on a different
CALL   MapAdcToCcp  ;page of program memory, so this makes sure
PAGESEL $           ;the CALL actually lands on it

BANKSEL CCP1H
MOVWF  ccp_next_h,W
MOVWF  CCP1H      ;manually load the very first pulse's end-time,
MOVWF  ccp_next_l,W ;since Timer1's ISR (which normally does this)
MOVWF  CCP1L      ;hasn't run yet
BSF    ADCDone,0  ;this value is ready and waiting

MOVLW  0b00000001 ;prescale 1:1, internal clock, and start Timer1
MOVWF  T1CON      ;counting right now, from 0x81E0

BSF    INTCON,6    ;allow peripherals to interrupt at all
BSF    INTCON,7    ;interrupts can now actually happen

Main:
BTFSC  Pulse,0     ;is a pulse still actively running
GOTO   Main        ;if high don't touch the ADC yet
BTFSC  ADCDone,0   ;have we already prepared next frame's value
GOTO   Main        ;if high then nothing more to do until the next frame
BSF    ADCCON0,1

ConvWait:
BTFSC  ADCON0,1
GOTO   ConvWait

BANKSEL ADRESH
MOVWF  ADRESH,W
MOVWF  adc_h      ;full 10-bit result, right-justified
BANKSEL ADRESL
MOVWF  ADRESL,W
BANKSEL adc_l
MOVWF  adc_l

BANKSEL ADCON0
PAGESEL MapAdcToCcp
CALL   MapAdcToCcp ;ccp_next_h:ccp_next_l now holds next
PAGESEL $          ;frame's pulse end-time

BSF    ADCDone,0   ;done for this frame - go idle
GOTO   Main

```

*Figure 25: New parts changed or added different from Part 3

- All other code used and schematic see pgs 117-119. The resolution in part 2 was 100µs per step, part 3 was about 32µs per step, and now in part 4 it is 2µs per step. The resolution isn't very noticable and using the frequency counter to measure PW is the best and a 10 turn pot is the only way to really see each step.

LG

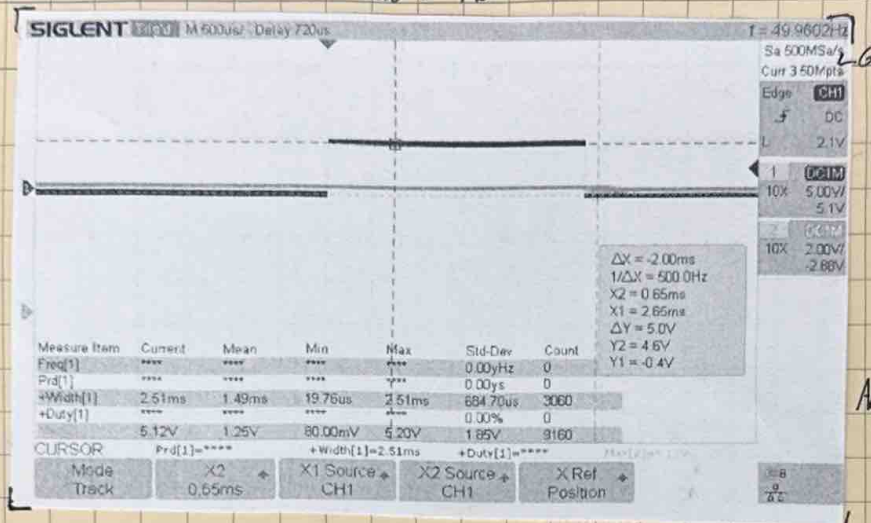


Figure 26: Oscilloscope Capture of Max PWM FCounter = (2513.919 μ s)

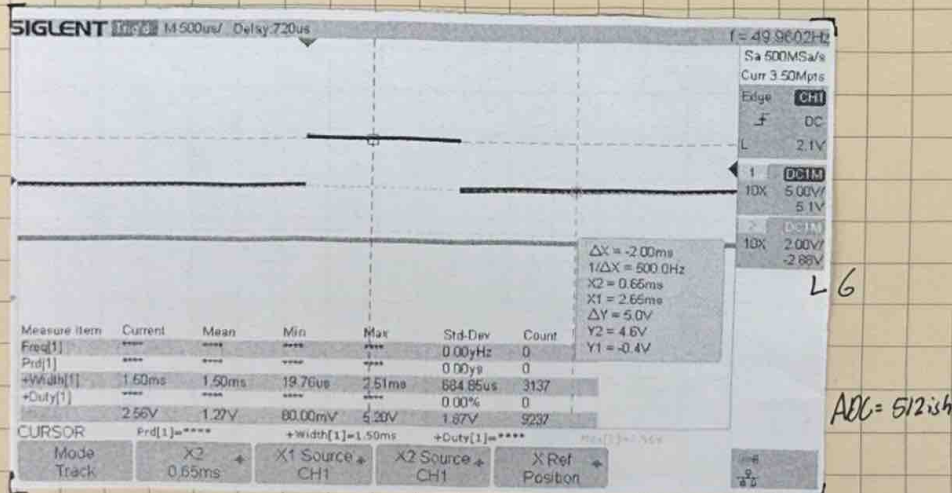


Figure 27: Oscilloscope capture of Middle PWM FCounter = 0499.956 μ s

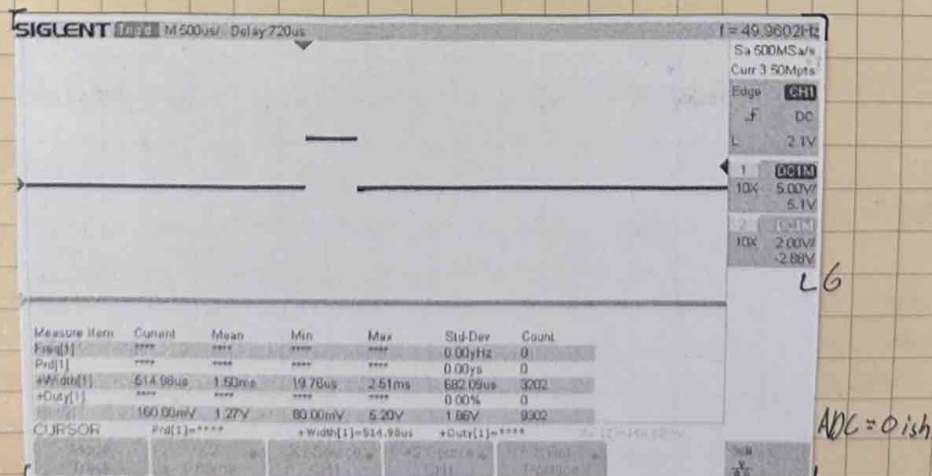


Figure 28: Oscilloscope Capture of Low PWM FCounter = (514.992 μ s)

^{LG}
~~Conclusion:~~ Conclusion:

- Lab7 ADC using the PIC16F883 to control a servo motor. ^{LG} During each part the resolution increased going from 100ps per step all the way to 2.1ps per step. Part1 was covering just proof of life, we could show the continuous converting using a potentiometer voltage and make the complete 10-bit resolution. The main thing to note here was all about knowing how to set up the code. Then by setting a diagnostic bit to see the interval of conversion. Part2 was mapping the ADC pulse for a resolution of 21 PW. that would range from 500ps to 2500ps. So each step would be about 100ps. See code and measurement on pg 112-115. Part3 moved the timing into hardware. Timer1 starts each 20ms frame and raises the servo pin. CCP1 Compare ends the pulse at a deadline from a 64-entry lookup table, and main prepares only one frame value per frame, after the pulse ends. Part4 replaced the table with a calculated mapping, giving it all 1024 commands about every 2ps per step.

- The trouble shooting was simple but annoying. it came down to making ^{LG} sure each register was in the right bank because ADRESH in bank 1 and ADRESH is in bank 0 which caused the issue in part 4 of seeing a weird resolution. Then RRF function making sure to clear the carry flag before hand.

TJR

10-9-26

LG